PAALHU: controllers still to build

Website API · 29 October 2026 · Based on the endpoint worksheet (Book.xlsx) · Built step by step in one file: `src/controllers/controller.js`

1. Where we are

#	Screen	Endpoint	Function	Login	Status
1	Menu	`GET /api/categories`	`getCategories`	No	**Done** commit 00afd6b
2	Menu	`GET /api/dishes`	`getDishes`	No	**To do**
3	Menu	`GET /api/drinks`	`getDrinks`	No	**To do**
4	Menu	`GET /api/menus`	`getMenus`	No	**To do**
5	Menu	`GET /api/menus/:id`	`getMenu`	No	**To do**
6	Login	`POST /api/clients`	`signUp`	No	**To do**
7	Login	`POST /api/auth/login`	`login`	No	**To do**
8	Booking	`GET /api/reservations/availability`	`getAvailability`	No	**To do**
9	Booking	`POST /api/reservations`	`createReservation`	Yes	**To do**
10	Booking	`PATCH /api/reservations/:id`	`changeReservationTime`	Yes	**To do**

1 of 10 done. The chat endpoints (`GET /api/chat/status`, `POST /api/chat`) and `GET /api/health` already exist and are not part of this list.

> **Agreed.** Only the 10 endpoints of the worksheet, all in one controller file, with one router (`src/routes/api.js`, mounted on `/api`). The last row of the worksheet, `PATCH /api/booking/:id`, becomes `PATCH /api/reservations/:id` with body `{ time }`. Each function is written and checked one at a time.

2. How every controller function works

Each function follows the same four steps:

1. **Read** the request: `req.query` (after the `?`), `req.params` (the `:id`) or `req.body` (the JSON sent).
2. **Validate** it with the rules of the worksheet. A wrong value throws a `ValidationError` (`src/validators.js`), which `errorHandler.js` turns into **400** `{ error, field }`.
3. **Use the data**: find, filter, save or change it.
4. **Answer** with `res.json(...)` and the right status: 200, 201 created, 401 not logged in, 404 not found, 409 conflict.

3. What each remaining function must do

Menu screen

2 · GET /api/dishes → getDishes	
Reads	`?category=&search=&allergen=&spice=&sort=&page=&limit=`
Does	Keeps dishes of that category id, whose name contains `search`, that do **not** contain `allergen`, with spice ≤ `spice`. Sorts (`-` = descending; default: category, then name) and returns one page.
Rules	page from 1 (default 1) · limit 1–100 (default 20) · spice 0–3 · allergen from a fixed list · sort only on known fields. Anything else → 400.
Answers	200 `{ data, page, limit, total, totalPages }`
Needs	The **dish data**: id, category id, name, price, spice 0–3, allergens. Prices from `pages/menu.html`. The `page()` and `limit()` validators already exist.

3 · GET /api/drinks → getDrinks	
Reads	`?search=&sort=&page=&limit=`
Does	Same as dishes, with only search, sort (default: by name) and paging.
Answers	200 `{ data, page, limit, total, totalPages }`
Needs	The 5 drinks with prices from the menu. Reuses the sorting and paging written for dishes.

4 · GET /api/menus → getMenus	
Reads	Nothing. Short, fixed list, no paging.
Answers	200 with the menus: À la carte and Set menu (€12.90, Monday to Friday, lunch only).

5 · GET /api/menus/:id → getMenu	
Reads	`:id` in the URL.
Does	Returns one menu with its contents, for the set menu pop-up: its four courses (starter, main, dessert or coffee, drink) and their options from `pages/set-menu.html`.
Answers	200 the menu · 400 id not a whole number · 404 no menu with that id.

Login screen

6 · POST /api/clients → signUp	
Reads	Body `{ name, email, phone, password }`
Rules	name 2–50 characters · valid email, not already registered · phone 9–15 digits, optional + · password at least 8 characters.
Does	Saves the client with the password **hashed** (never in plain text).
Answers	201 the client without the password · 400 invalid field · 409 email already registered.
Needs	A clients list, and password hashing (Node's built-in `crypto`, no extra package).

7 · POST /api/auth/login → login	
Reads	Body `{ email, password }`
Rules	valid email · password not empty and matching the stored one.
Answers	200 `{ token, client }` · 400 invalid field · 401 wrong email or password (same message for both).
Needs	A way to give and check tokens, and a small **login check** run before functions 9 and 10: it reads `Authorization: Bearer <token>` and answers 401 if it is missing or wrong.

Booking screen

These three use the booking engine that already exists (`.claude/skills/paalhu-reservas/reservas-core.mjs`): tables, shifts, joining tables and free-table checks. It is the same register the chat and the staff use, so a table booked on the website is taken for everyone.

8 · GET /api/reservations/availability → getAvailability	
Reads	`?date=&guests=`, both required.
Does	Lists the arrival times of that day that still have a free table for that many guests.
Answers	200 the free times (empty when closed) · 400 missing or invalid date or guests (1–10).

9 · POST /api/reservations → createReservation (login)	
Reads	Body `{ date, time, guests }`. Name and phone come from the logged-in client.
Rules	date YYYY-MM-DD, today or later, not a closed day · time HH:MM and one of that day's times · guests 1–10 and a table must be free.
Answers	201 the reservation · 400 invalid field or closed day · 401 not logged in · 409 no free table at that time.
Needs	The engine must remember which client made the reservation, so function 10 can check it.

10 · PATCH /api/reservations/:id → changeReservationTime (login)	
Reads	`:id` (like R-20261114-001) and body `{ time }`.
Does	Moves the client's own reservation to another time on the same day.
Answers	200 the changed reservation · 400 invalid time · 401 not logged in · 404 not found or not this client's · 409 cancelled, or no free table at the new time.

4. Decisions still open

Topic	Question	Affects
Allergens	The restaurant does not publish allergens. Real data from the restaurant, a marked draft, or empty for now? A wrong allergen is a health risk.	2
Spice 0–3	The worksheet filters on 0–3, but customers choose 1–5 (spice guide). Proposal: 0–3 is how hot the dish comes; 1–5 stays the level asked for when ordering.	2
Where data lives	Menu data and clients at the top of `controller.js` (like the categories), or in a separate data file? Clients kept in memory are lost when the server restarts.	2–7
Reservation times	Which times count as "available slots": every 15 minutes inside each shift of `config.json`?	8–10

Today	The worksheet allows booking for today; the chat only allows from tomorrow. Keep them different?	8, 9
Group size	The worksheet says 1–10 guests; the booking engine allows up to 12. Use 10 on the website?	8, 9

5. Suggested order

1. **getDishes**, then **getDrinks** (same pattern, reuse the paging).
2. **getMenus** and **getMenu** (fixed data, quick).
3. **signUp** and **login**, plus the login check.
4. **getAvailability**, then **createReservation** and **changeReservationTime**.

Each step: write one function, add its line to `src/routes/api.js`, try it in the browser or Postman, commit.